DEPARTMENT OF COMPUTER APPLICATIONS

PSG COLLEGE OF TECHNOLOGY

MCA G1 & G2, SEMESTER 1

26MX16 - PROGRAMMING AND DATA STRUCTURES LABORATORY

PRACTICE SHEET 7

Prerequisite	Basics of C programming, arrays, functions, loops, conditional statements, pointers and basic problem-solving skills.

1. A smart parking facility maintains vehicles in a narrow parking lane where vehicles are parked one behind another. Since only the most recently parked vehicle can leave first, the system follows the LIFO (Last In, First Out) principle. Write a program to implement the same system using a stack as per the following:

1. Push vehicle registration numbers into the stack.
2. Pop a vehicle when it exits the parking lane.
3. Display the vehicle currently at the top.
4. Display all vehicles currently parked.
5. Search whether a given vehicle is currently present in the parking stack.
6. Handle stack overflow and stack underflow conditions.

Test Case 1: 1) Push 2) Pop 3) Peek 4) Display 5) Search

Input	**Output**	
Option: 1	Vehicle TN01AB1234	Vehicle TN02CD5678 is
TN01AB1234	pushed into the stack.	present in the parking stack.
Option: 1	Vehicle TN02CD5678	Vehicle TN03EF9012
TN02CD5678	pushed into the stack.	removed from the stack.
Option: 1	Vehicle TN03EF9012	Top vehicle: TN02CD5678
TN03EF9012	pushed into the stack.	
Option: 3	Top vehicle: TN03EF9012	
Option: 4	Vehicles currently parked:	
Option: 5	TN03EF9012	
Option: 2	TN02CD5678	
Option: 3	TN01AB1234	

Test Case 2: 1) Push 2) Pop 3) Peek 4) Display 5) Search

Input	2	Stack Overflow! Parking
1	2	stack is full.
TN10AA1111	2	
1	2	Vehicle TN50EE5555
TN20BB2222	**Output**	removed from the stack.
1	Vehicle TN10AA1111	Vehicle TN40DD4444
TN30CC3333	pushed into the stack.	removed from the stack.
1	Vehicle TN20BB2222	Vehicle TN30CC3333
TN40DD4444	pushed into the stack.	removed from the stack.
1	Vehicle TN30CC3333	Vehicle TN20BB2222
TN50EE5555	pushed into the stack.	removed from the stack.
1	Vehicle TN40DD4444	Vehicle TN10AA1111
TN60FF6666	pushed into the stack.	removed from the stack.
2	Vehicle TN50EE5555	
2	pushed into the stack.	Stack Underflow! No vehicle
2		is present.

2. A web browser maintains the pages visited by a user using a stack. When the user selects Back, the most recently visited page is removed from the stack. Write a program to implement the same using a stack as per the following:

1. Push a newly visited URL into the stack.
2. Perform the back operation using pop().
3. Display the current page using peek().
4. Display browsing history from the top of the stack.
5. Search for a particular URL in the browsing history.
6. Display an appropriate message when the user tries to go back when the stack is empty.

Test Case 1: Visit Pages, Back, Peek and Search

Input	**Output**	
1 https://google.com 1 https://youtube.com 1 https://github.com 3 4 5 https://youtube.com 2 3	URL https://google.com visited and pushed into the stack. URL https://youtube.com visited and pushed into the stack. URL https://github.com visited and pushed into the stack. Current page: https://github.com	Browsing history: https://github.com https://youtube.com https://google.com URL https://youtube.com is present in the browsing history. Back operation: https://github.com removed. Current page: https://youtube.com

Test Case 2: Back Operation on Empty Stack

Input	
1 https://example.com 2 2 3	Back operation: https://example.com removed. No back operation is possible. Browsing history is empty. No current page available.
Output URL https://example.com visited and pushed into the stack.	

3. A compiler checks whether brackets in a source-code expression are properly balanced. The brackets (), {}, and [] must follow the correct nesting order. Write a program to implement the same using a stack as per the following:

1. Read an expression containing brackets and other characters.
2. Push every opening bracket onto the stack.
3. When a closing bracket is encountered, pop the corresponding opening bracket.
4. Verify whether the brackets match.
5. Display:
    - Balanced Expression
    - Unbalanced Expression
6. Handle cases where:
    - The stack becomes empty before a matching opening bracket is found;
    - Opening brackets remain after scanning the expression.

Test Case 1: Balanced Expression **Input** {[a+(b*c)]-d} **Output** Balanced Expression **Test Case 2: Unbalanced Expression** **Input** {[a+(b*c)]-d	**Output** Unbalanced Expression Opening brackets remain in the stack. **Test Case 3: Incorrect Nesting** **Input** ([)] **Output** Unbalanced Expression	Mismatched brackets found. **Test Case 4: Closing Bracket Without Opening Bracket** **Input** a+b) **Output** Unbalanced Expression No matching opening bracket found for ')'.

4. A text editor maintains the user's recent modifications using a stack. Every modification is stored in the stack, and the undo operation removes the most recent modification. Write a program to implement the same a stack as per the following:

1. Push every new editing operation into the stack.
2. Perform Undo using pop().
3. Display the most recent operation using peek().
4. Display all available undo operations.
5. Search for an operation using its Operation ID.
6. Display "No undo operation is available", when the stack is empty.

Test Case 1: Add, Undo, Peek and Search

Input

1
101 Insert Text
1
102 Delete Word
1
103 Change Font
3
4
5
102
2
3

Output

Operation 101 Insert Text pushed into the stack.
Operation 102 Delete Word pushed into the stack.
Operation 103 Change Font pushed into the stack.

Most recent operation: 103 Change Font

Available undo operations:
103 Change Font
102 Delete Word
101 Insert Text

Operation ID 102 found in the undo stack.

Undo operation: 103 Change Font removed.
Most recent operation: 102 Delete Word

Test Case 2: Empty Undo Stack

Input

2
3
4
5
101

Output

No undo operation is available.
No undo operation is available.
No undo operation is available.
Operation ID 101 not found.

---

5. A calculator internally evaluates arithmetic expressions using different representations. The expression contains single-digit operands and the operators: +, -, *, /. Write a program to implement the same a stack as per the following:

1. Read a postfix expression.
2. Scan the expression from left to right.
3. Push operands onto the stack.
4. When an operator is encountered:
    - Pop the required operands;
    - Perform the operation;
    - Push the result back onto the stack.
5. Display the final result.
6. Handle:
    - Insufficient operands;
    - Division by zero;
    - Invalid symbols;
    - Empty expression.

DEPARTMENT OF COMPUTER APPLICATIONS
PSG COLLEGE OF TECHNOLOGY
MCA G1 & G2, SEMESTER 1

Test Case 1: Valid Postfix Expression
Input
23*54*+
Output
Final Result: 26
Explanation:
2 3 * = 6
5 4 * = 20
6 + 20 = 26
Test Case 2: Division and Multiplication
Input
82/3-
Output
Final Result: 1
Explanation:
8 / 2 = 4
4 - 3 = 1
Test Case 3: Division by Zero
Input
50/
Output
Error: Division by zero.
Test Case 4: Insufficient Operands
Input
5+
Output
Error: Insufficient operands for operator '+'.
Test Case 5: Invalid Symbol
Input
23a+
Output
Error: Invalid symbol 'a'.

Skills Acquired:	**Stack operations: Push, Pop, Peek, Overflow and Underflow, Parenthesis matching, Expression conversion and evaluation.**